

```
=====
PANDAS COMPLETE CHEAT SHEET
=====
```

```
=====
1. IMPORT
=====
```

```
import pandas as pd
import numpy as np
```

```
=====
2. CSV / EXCEL LOAD & SAVE
=====
```

```
# CSV load karo
df = pd.read_csv("file.csv")
```

```
# Encoding issue aaye to
df = pd.read_csv("file.csv", encoding='latin1')
```

```
# Sirf kuch columns chahiye
df = pd.read_csv("file.csv", usecols=["col1", "col2", "col3"])
```

```
# Sirf pehli 100 rows
df = pd.read_csv("file.csv", nrows=100)
```

```
# CSV save karo
df.to_csv("output.csv", index=False)          # index=False matlab row number column
nahi aayega
```

```
# Excel load karo
df = pd.read_excel("file.xlsx")
```

```
# Excel ki specific sheet
df = pd.read_excel("file.xlsx", sheet_name="Sheet1")
```

```
# Excel save karo
df.to_excel("output.xlsx", index=False)
```

```
=====
```

3. DATAFRAME KI BASIC INFO

```
=====

df.shape                # (rows, columns) - kitni rows aur columns hain
df.columns.tolist()     # sare column names list me
df.dtypes               # har column ka data type
df.head(5)              # pehli 5 rows
df.tail(5)              # aakhiri 5 rows
df.info()               # ek saath shape + dtypes + null count
df.describe()           # numeric columns ka min/max/mean/std
df.sample(10)           # random 10 rows
len(df)                 # total row count
df.nunique()             # har column me unique values ki count
df['col'].unique()       # ek column ki unique values
df['col'].value_counts() # har unique value kitni baar aai

=====
```

4. COLUMN SELECT KARO

```
=====

# ek column - Series aata hai
df['name']

# ek column - DataFrame chahiye
df[['name']]

# multiple columns
df[['name', 'age', 'city']]

# column rename karo
df = df.rename(columns={'purana_naam': 'naya_naam'})

# multiple rename
df = df.rename(columns={'col1': 'a', 'col2': 'b'})

# column drop karo
df = df.drop(columns=['col1', 'col2'])

# column ka naam change karo in place
df.columns = ['a', 'b', 'c']          # sare columns ek saath rename

=====
```

5. ROWS FILTER KARO (CONDITIONS)

```

# ek condition
df[df['age'] > 18]

# do conditions - AND
df[(df['age'] > 18) & (df['gender'] == 1)]

# do conditions - OR
df[(df['cat1'] == 0) | (df['cat1'] == 9)]

# list me se check karo - isin()
df[df['cat1'].isin([0, 1, 2])]

# list me se nahi - ~isin()
df[~df['cat1'].isin([0, 1, 2])]

# null rows
df[df['col'].isnull()]

# non-null rows
df[df['col'].notnull()]

# string contains
df[df['name'].str.contains('Kumar')]

# string exact match (case insensitive)
df[df['name'].str.lower() == 'rahul']

# row number se - loc aur iloc
df.loc[0]          # index 0 wali row
df.iloc[0]         # position 0 wali row
df.iloc[0:5]       # pehli 5 rows
df.loc[df['age'] > 18] # condition se rows

```

6. NULL / MISSING VALUES

```

# kul nulls har column me
df.isnull().sum()

# kisi column me null hai ya nahi check
df['col'].isnull().any()          # True/False

# null fill karo
df['col'].fillna(0)                # number se
df['col'].fillna('')              # blank string se

```

```

df['col'].fillna(df['col'].mean()) # mean se
df['col'] = df['col'].fillna(method='ffill') # upar wali value se

# null rows drop karo
df = df.dropna() # koi bhi null ho
df = df.dropna(subset=['col1']) # sirf col1 null ho tab

# null replace karo condition se
mask = df['col'].isnull()
df.loc[mask, 'col'] = 'DEFAULT'

# pd.NA aur np.nan me farak
# pd.NA - nullable integer ke liye
# np.nan - float columns ke liye
# dono isnull() se pakad aate hain

```

```

=====
7. COLUMN BANAO / BADLE
=====

```

```

# naya column fixed value
df['new_col'] = 0

# naya column calculation se
df['total'] = df['score'] + df['bonus']

# naya column condition se - np.where
df['pass_fail'] = np.where(df['score'] >= 40, 'Pass', 'Fail')

# naya column multiple conditions se - np.select
conditions = [
    df['score'] >= 60,
    df['score'] >= 40,
    df['score'] < 40
]
choices = ['A', 'B', 'C']
df['grade'] = np.select(conditions, choices, default='Unknown')

# column ko int/float/str me convert karo
df['col'] = df['col'].astype(int)
df['col'] = df['col'].astype(float)
df['col'] = df['col'].astype(str)

# float ko int - safely (NaN ho sakta hai)
df['col'] = df['col'].astype('Int64') # capital I - nullable integer

```

```
# float string '2.0' ko int string '2' me
df['col'] = df['col'].astype(float).astype(int).astype(str)

# string column se number nikalo
df['col'] = pd.to_numeric(df['col'], errors='coerce') # error aaye to NaN

# date column banao
df['dob'] = pd.to_datetime(df['dob'])
df['dob'] = pd.to_datetime(df['dob'], format='%d/%m/%Y') # custom format
```

8. STRING OPERATIONS

```
df['col'].str.upper() # UPPERCASE
df['col'].str.lower() # lowercase
df['col'].str.strip() # aage peeche space hatao
df['col'].str.len() # string ki length
df['col'].str.replace('a', 'b') # replace karo
df['col'].str.replace('9', '', regex=False) # exactly '9' hatao
df['col'].str.contains('Kumar') # contains check
df['col'].str.startswith('A') # starts with check
df['col'].str.split(',') # split karo
df['col'].str.split(',').str[0] # split ke baad pehla element
df['col'].str.zfill(2) # '1' → '01', left side zeros
df['col'].str.extract(r'(\d+)') # regex se nikalo
```

9. DATE OPERATIONS

```
df['dob'] = pd.to_datetime(df['dob'])

df['dob'].dt.year # year nikalo
df['dob'].dt.month # month nikalo
df['dob'].dt.day # day nikalo

# date comparison
df[df['dob'] > pd.to_datetime('2002-01-01')]

# date difference
df['age_days'] = pd.to_datetime('today') - df['dob']

# date add karo
df['new_date'] = df['dob'] + pd.DateOffset(years=5)
```

```
df['new_date'] = df['dob'] + pd.DateOffset(years=3, months=2, days=10)
```

```
=====
```

```
10. SORT KARO
```

```
=====
```

```
# ascending
```

```
df = df.sort_values(by='merit')
```

```
# descending
```

```
df = df.sort_values(by='score', ascending=False)
```

```
# multiple columns - alag alag direction
```

```
df = df.sort_values(  
    by=['total_marks', 'parta_gi', 'dob'],  
    ascending=[False, False, True]  
)
```

```
# index reset karo sort ke baad
```

```
df = df.sort_values(by='merit').reset_index(drop=True)
```

```
=====
```

```
11. GROUPBY
```

```
=====
```

```
# group karke count
```

```
df.groupby('cat1').size()
```

```
# group karke sum
```

```
df.groupby('state')['vacancies'].sum()
```

```
# group karke mean
```

```
df.groupby('gender')['score'].mean()
```

```
# group karke multiple aggregations
```

```
df.groupby('cat1').agg(  
    total_candidates=('rollno', 'count'),  
    avg_score=('score', 'mean'),  
    max_score=('score', 'max')  
)
```

```
# groupby ke baad reset index
```

```
df.groupby('cat1').size().reset_index(name='count')
```

```
=====
12. MERGE / JOIN (do dataframes milao)
=====
```

```
# inner join - dono me hona chahiye
merged = pd.merge(df1, df2, on='rollno', how='inner')

# left join - df1 ki sari rows rahegi
merged = pd.merge(df1, df2, on='rollno', how='left')

# right join
merged = pd.merge(df1, df2, on='rollno', how='right')

# alag column names pe join
merged = pd.merge(df1, df2, left_on='rollno', right_on='roll_no', how='left')

# multiple columns pe join
merged = pd.merge(df1, df2, on=['state', 'gender'], how='left')
```

```
=====
13. APPLY - ROW/COLUMN PE FUNCTION CHALAO
=====
```

```
# ek column pe function apply karo
df['col'] = df['col'].apply(lambda x: x * 2)

# row pe function apply karo (axis=1)
df['result'] = df.apply(lambda row: row['a'] + row['b'], axis=1)

# complex logic - function banao aur apply karo
def calculate_flag(row):
    if row['score'] >= 60:
        return 'A'
    elif row['score'] >= 40:
        return 'B'
    else:
        return 'C'

df['grade'] = df.apply(calculate_flag, axis=1)

# sirf filtered rows pe apply karo
mask = df['merit'].notnull()
df.loc[mask, 'grade'] = df[mask].apply(calculate_flag, axis=1)
```

14. UPDATE – EK DF SE DUSRE ME UPDATE KARO

```
=====
# df2 ki values df1 me update karo (same index chahiye)
df1.update(df2)
```

```
# specific rows update karo loc se
df.loc[df['cat1'] == 0, 'label'] = 'UR'
```

```
# mask se update
mask = df['score'] > 100
df.loc[mask, 'score'] = 100
```

```
# ek column ki value dusri column se
df.loc[mask, 'col1'] = df.loc[mask, 'col2']
```

15. DUPLICATE CHECK & REMOVE

```
=====
# duplicates check karo
df.duplicated().sum()                # kul duplicate rows
df.duplicated(subset=['rollno']).sum() # rollno pe duplicate
```

```
# duplicate rows dikho
df[df.duplicated(subset=['rollno'])]
```

```
# duplicates drop karo
df = df.drop_duplicates(subset=['rollno'])
df = df.drop_duplicates(subset=['rollno'], keep='first') # pehla rakho
df = df.drop_duplicates(subset=['rollno'], keep='last')  # aakhiri rakho
```

16. COPY

```
=====
# SAHI TARIKA – deep copy
cand = candidate_df.copy()
```

```
# GALAT TARIKA – dono ek hi object point karenge
cand = candidate_df      # ek me change dono me hoga
```

17. MASK PATTERN (Jo baar baar use hota hai)


```
=====

# null wali rows ko kuch do, non-null ko kuch aur
mask = df['col'].isnull()
df.loc[mask, 'col'] = 'UNKNOWN'
df.loc[~mask, 'col'] = df.loc[~mask, 'col'].astype(str).str.strip()

# condition pe ek column ki value dusri column se set karo
mask = df['cat1'].isin([1, 2])
df.loc[mask, 'label'] = 'Reserved'
```

=====

18. DICTIONARY SE COLUMN MAP KARO

=====

```
# dictionary banao
cat_names = {0: 'UR', 1: 'SC', 2: 'ST', 6: 'EWS', 9: 'Migrated'}

# map karo - missing values NaN honge
df['cat_name'] = df['cat1'].map(cat_names)

# missing values ke liye default
df['cat_name'] = df['cat1'].map(cat_names).fillna('Unknown')

# set_index + map shortcut (lookup table)
rollno_to_merit = candidates.set_index('rollno')['merit'].to_dict()
df['merit'] = df['rollno'].map(rollno_to_merit)
```

=====

19. VALUE COUNTS & DISTRIBUTION

=====

```
# simple count
df['col'].value_counts()

# percentage
df['col'].value_counts(normalize=True) * 100

# nulls bhi dikhao
df['col'].value_counts(dropna=False)

# dictionary me convert karo
df['col'].value_counts().to_dict()

# cross tabulation - do columns ka relation
```

```
pd.crosstab(df['gender'], df['cat1'])
```

20. SHAPE / COUNT SHORTCUTS

```
df.shape[0]                # total rows
df.shape[1]                # total columns
df[df['col'] > 0].shape[0]  # condition wali rows ki count
df['col'].isnull().sum()    # null count
df['col'].notnull().sum()   # non-null count
(df['col'] == 'Yes').sum()  # specific value count
df[df['allocated'].notnull()].shape[0]  # allocated candidates count
```

21. CONCAT – DO DATAFRAMES JODO

```
# rows jodo (upar neeche)
df_all = pd.concat([df1, df2], ignore_index=True)

# columns jodo (side by side)
df_all = pd.concat([df1, df2], axis=1)

# list of dataframes jodo
df_all = pd.concat([df1, df2, df3, df4], ignore_index=True)
```

22. LAMBDA QUICK REFERENCE

```
# simple
lambda x: x * 2

# condition
lambda x: 'Yes' if x > 0 else 'No'

# string operation
lambda x: str(x).zfill(2)

# row pe (axis=1)
lambda row: row['a'] + row['b']

# try-except inside lambda – nahi karo, function banao
```

```
=====
23. REGEX – str.extract & str.match
=====
```

```
# pattern se values nikalo
df[['yrs', 'months', 'days']] = df['service'].str.extract(
    r'(\d+) Years (\d+) Months (\d+) Days'
)

# pattern match karo – True/False
df['valid'] = df['col'].str.match(r'\d{10}')    # 10 digit number

# pattern find karo
df['col'].str.contains(r'\d+')

=====
```

```
24. COMMON ERRORS AUR FIX
=====
```

```
# SettingWithCopyWarning – filtered df me directly set kiya
# GALAT
filtered = df[df['col'] > 0]
filtered['new'] = 1

# SAHI
filtered = df[df['col'] > 0].copy()
filtered['new'] = 1

# ya directly loc se
df.loc[df['col'] > 0, 'new'] = 1

# -----

# KeyError: 'column_name' – column exist nahi karta
print(df.columns.tolist())    # pehle check karo

# -----

# ValueError: float NaN cannot be converted to int
# GALAT
df['col'].astype(int)

# SAHI
df['col'].astype('Int64')    # nullable integer
```

```

# ya pehle fill karo
df['col'].fillna(0).astype(int)

# -----

# TypeError: '>' not supported between str and int
# Column str me hai, int chahiye
df['col'] = pd.to_numeric(df['col'], errors='coerce')

# -----

# Slow code – row by row loop mat karo
# GALAT (bahut slow)
for i, row in df.iterrows():
    df.loc[i, 'col'] = row['a'] + row['b']

# SAHI (fast)
df['col'] = df['a'] + df['b']

=====
25. USEFUL ONE-LINERS
=====

# sabse zyada score wali row
df.loc[df['score'].idxmax()]

# har group me max merit wali row
df.loc[df.groupby('state')['merit'].idxmax()]

# column exist karta hai ya nahi
'col_name' in df.columns

# koi bhi null hai ya nahi
df.isnull().any().any()

# har column me null percentage
(df.isnull().sum() / len(df) * 100).round(2)

# do columns ka difference
df['diff'] = df['col1'] - df['col2']

# cumulative sum
df['running_total'] = df['score'].cumsum()

# rank
df['rank'] = df['score'].rank(ascending=False, method='min')

```

```
# index reset
df = df.reset_index(drop=True)

# column ko int karo safely jab float strings ho ('1.0', '2.0')
df['col'] = df['col'].astype(float).astype(int)

# multiple columns ek saath None karo
df[['a', 'b', 'c']] = None
```

```
=====
                        END OF CHEAT SHEET
=====
```